

AI Orbit Data Ingestion Pipeline Architecture

1. System Architecture

The AI Orbit Ingestion Pipeline is an asynchronous, large-scale data discovery, extraction, and resolution system that covers 10 distinct modules (Companies, Tools, Agents, MCP, Models, Robots, Devices, Papers, News, Jobs).

- **Source Adapters** (`discovery/`): Pluggable components tailored to specific endpoints (e.g., arXiv RSS, TechCrunch RSS, HuggingFace API, GitHub Search API, Remote API).
- **Crawling Engine** (`crawling/`): Asynchronous HTTP fetching via `aiohttp` using semaphores to cap maximum concurrent connections and prevent blocking.
- **LLM Orchestration** (`llm/`): Utilizes `instructor` with Pydantic schemas to strictly enforce extraction structures. Orchestrator supports fallback logic: OpenAI -> Gemini -> Groq -> DeepSeek.
- **Storage** (`storage/`): Outputs normalized records to standard JSON files and Google Sheet-ready CSVs.

2. Reliability & Resilience

- **429 Handling**: The `async_retry` decorator dynamically responds to HTTP 429 Too Many Requests by applying exponential backoff and jitter to prevent thundering herd problems.
- **413 Payload Too Large**: Implements intelligent text chunking (`chunking.py`) that separates large HTML/PDF text bodies into paragraph-aware chunks to fit within LLM token/payload limits.
- **Anti-Bot Strategy** (Cloudflare/JS): Graceful degradation. If an endpoint blocks automated access via 403 (like OpenAI or Perplexity), it logs the failure and falls back to other sources like GitHub APIs or RSS feeds. The architecture natively supports Playwright browser extensions if JS execution is required.
- **Freshness Limits**: News and Jobs adapters strictly enforce a `<24h` collection freshness limit by normalizing timestamps and checking against UTC bounds.

3. Scaling to 500k+ Records

The current Python/asyncio architecture forms the foundation. For 500k+ records, the design scales without rewriting the core pipeline via:

- **Message Queues**: RabbitMQ or Kafka replacing in-memory `asyncio.gather` for URL dispatching.
- **Partitioned Workers**: Distributed Celery/Temporal workers executing source-specific crawlers.
- **Incremental Crawling & Checkpointing**: Persisting last-seen dates to fetch only new/updated records, combined with deterministic hashing to ensure idempotency.
- **Database Strategy**:
 - Relational storage (PostgreSQL) for canonical entities and pagination.
 - Vector storage (pgvector or Pinecone) for semantic deduplication (fuzzy match replacement).
 - Graph Database (Neo4j) to query nested relationships (e.g., Company -> DEVELOPS -> Tool).

4. Entity Resolution & Distributed Deduplication

Entities pass through a 6-step deterministic resolution (``entity_resolution/resolver.py``):

1. Canonical URL match
2. Normalized Name match
3. Alias match
4. External ID mapping
5. Fuzzy matching ratio
6. Manual review logging

All merges and derivations are tracked in an ``Entity_Mapping_Log.csv`` to ensure high traceability and data provenance.